Especificación de Requisitos de Software

HuertoHogar · Versión 2.0 · Evaluación Parcial 2

Asignatura: DSY1104, Desarrollo Fullstack II. Equipo: Sebastian Monsalve y Lucas Parra. Actualización: 21 de octubre de 2026.

1. Propósito y antecedentes

Actualizar HuertoHogar desde HTML, CSS y JavaScript a una aplicación frontend con React, preservando el catálogo, los recursos multimedia y la temática de productos frescos. La versión actual incorpora componentes reutilizables, navegación, formularios controlados, ofertas, proceso de compra simulado, administración y pruebas automatizadas con Jasmine y Karma.

Este ERS describe la versión desarrollada y comprobada para la Evaluación Parcial 2. Los pagos, despachos y sesiones funcionan como simulaciones locales; no hay backend.

2. Alcance

El visitante consulta productos, filtra por categoría, revisa ofertas y utiliza el carrito. Puede registrar una cuenta de prueba, iniciar sesión y enviar mensajes de contacto. El cliente puede ingresar al checkout, completar sus datos, elegir entrega simulada y obtener un resultado exitoso o fallido.

En una compra simulada exitosa se guarda el pedido en localStorage, se descuenta el stock y se vacía el carrito. En un pago simulado fallido no se descuenta stock y se ofrece reintento. El administrador gestiona productos, descuentos, usuarios y consulta el historial de pedidos locales.

3. Actores y entorno

Visitante: navega, consulta productos, ofertas, formularios y carrito. Cliente: usuario registrado con sesión local y datos reutilizables para el checkout. Administrador: utiliza el panel de productos, usuarios y pedidos. Desarrollador/evaluador: instala dependencias, ejecuta pruebas y revisa cobertura.

Entorno: navegador moderno, React con Vite, Bootstrap, Node.js y npm para desarrollo y testing, Jasmine/Karma con Chrome Headless y almacenamiento local del navegador.

4. Requisitos funcionales y criterios de aceptación

ID	Requisito	Criterio de aceptación	Implementación
RF01	Navegar entre secciones	Rutas sin recarga y aviso ante ruta desconocida	Aplicacion, Layout, Contenido
RF02	Consultar catálogo	Productos iniciales con nombre, imagen, precio y stock	Catalogo, TarjetaProducto
RF03	Filtrar y buscar	Filtro de categoría y texto combinables	Catalogo
RF04	Consultar detalle	Detalle por código; aviso si no existe	Detalle
RF05	Agregar al carrito	Cantidades enteras positivas que no superen el stock	Tienda, carrito
RF06	Mantener carrito	Cambiar cantidades, eliminar, vaciar y calcular total	Carrito
RF07	Persistir estado	Carrito, catálogo y sesión local al recargar	almacenamiento, Tienda
RF08	Registrar cuenta	Validar RUN módulo 11 y duplicados, correo, contraseña confirmada, dirección, región y comuna; mostrar errores específicos	FormularioCuenta, validaciones

ID	Requisito	Criterio de aceptación	Implementación
RF09	Iniciar/cerrar sesión	Credenciales de prueba y cierre local de sesión	FormularioCuenta, Estructura
RF10	Contactar	Validar datos y guardar mensaje local	FormularioCuenta
RF11	Mostrar contenido	Inicio con video, Nosotros, blog y artículos	Inicio, Contenido
RF12	Administrar productos	Crear, editar y eliminar; controlar stock, código, precio y descuento	Administracion
RF13	Administrar usuarios	Consultar cuentas, cambiar roles y eliminar	Administracion
RF14	Continuar al checkout	Carrito no vacío conduce al formulario sin borrar productos	Carrito, Checkout
RF15	Consultar ofertas	Mostrar productos con descuento y precio rebajado	Ofertas, TarjetaProducto
RF16	Validar checkout	Mensajes individuales para nombre, correo, dirección, región y comuna	Checkout, validaciones
RF17	Simular resultado del pago	Resultado exitoso o fallido; permitir reintento en caso de fallo	Checkout, Resultado
RF18	Registrar compra simulada	En éxito guardar pedido, descontar stock y vaciar carrito; en fallo conservarlo	Checkout, Tienda
RF19	Consultar pedidos	Admin visualiza historial local de compras simuladas	Administracion

5. Reglas de negocio

Productos: código único de 3 a 12 caracteres alfanuméricos mayúsculos; precio mayor que cero; stock entero no negativo; descuento entero entre 0 y 90%. Los descuentos visibles se aplican al precio cobrado. El carrito rechaza cantidades que exceden existencias.

Cuentas: nombre de 2 a 100 caracteres; correos permitidos @gmail.com, @duoc.cl y @profesor.duoc.cl, sin doble @, espacios ni puntos consecutivos en el nombre; RUN con dígito verificador válido y sin duplicados; contraseña de 4 a 10 caracteres y confirmación coincidente. Región y comuna deben pertenecer a las opciones del prototipo; la dirección debe contener información utilizable, con número o indicación S/N.

Compra: los pagos son simulados. Solo una simulación exitosa registra el pedido y reduce stock. No existe cobro, despacho ni transferencia de información a servicios externos. El contacto almacena mensajes locales y no envía correo real.

6. Requisitos no funcionales

ID	Requisito	Evidencia o criterio
RNF01	React modular	Componentes funcionales, props, useState, useEffect y Context
RNF02	Bootstrap responsivo	Grillas adaptables y navegación móvil
RNF03	Accesibilidad básica	Etiquetas, avisos con role y mensajes de error cercanos al campo
RNF04	Pruebas reproducibles	npm.cmd install, npm.cmd run build, npm.cmd test
RNF05	Cobertura mínima	Sentencias/líneas >=90%; ramas/funciones >=85%
RNF06	Manejo de almacenamiento	Fallback si localStorage falla o contiene JSON corrupto
RNF07	Hosting estático	HashRouter con rutas compatibles con despliegue estático

7. Arquitectura y flujo de datos

Aplicacion configura HashRouter y las rutas. Estructura reúne navegación, avisos y pie. Tienda actúa como proveedor de estado común para productos, usuario y carrito. Catalogo, Ofertas, Detalle y TarjetaProducto muestran productos; carrito.js calcula precios, cantidades y total. FormularioCuenta y Checkout utilizan reglas de validaciones.js. El estado local se persiste mediante almacenamiento.js.

La compra simulada crea un registro local de pedido, actualiza el inventario y navega a Resultado. Administracion permite gestionar productos, descuentos, usuarios y consultar pedidos. La interfaz no consulta un servidor.

8. Restricciones y riesgos conocidos

La información pertenece a un solo navegador y origen y no se sincroniza entre dispositivos. localStorage puede ser modificado mediante herramientas del navegador y no sirve para autenticación de producción. Las contraseñas de demostración están almacenadas localmente. Los cambios de inventario o precio pueden requerir actualización del carrito para evitar datos desactualizados.

No existe garantía de persistencia si el navegador bloquea o agota el almacenamiento. La solución no incluye pagos reales, envíos, correo real, base de datos ni control de acceso de servidor.

9. Verificación y entrega

El 21 de octubre de 2026 se compiló correctamente con Vite y se ejecutaron 58 pruebas automatizadas: 58 exitosas y 0 fallidas. Cobertura global: sentencias 90,07% (363/403); ramas 87,04% (336/386); funciones 86,86% (119/137); líneas 94,32% (299/317). Los cuatro valores superan los objetivos definidos en Karma.

La revisión visual anterior de ocho rutas a 390, 768 y 1440 píxeles corresponde a una versión previa; las nuevas rutas de checkout y ofertas requieren comprobación manual complementaria. La entrega contiene el código fuente, configuración de Vite/Karma, tests, recursos y los documentos ERS y cobertura. Los reportes regenerables de coverage no son necesarios dentro del repositorio.